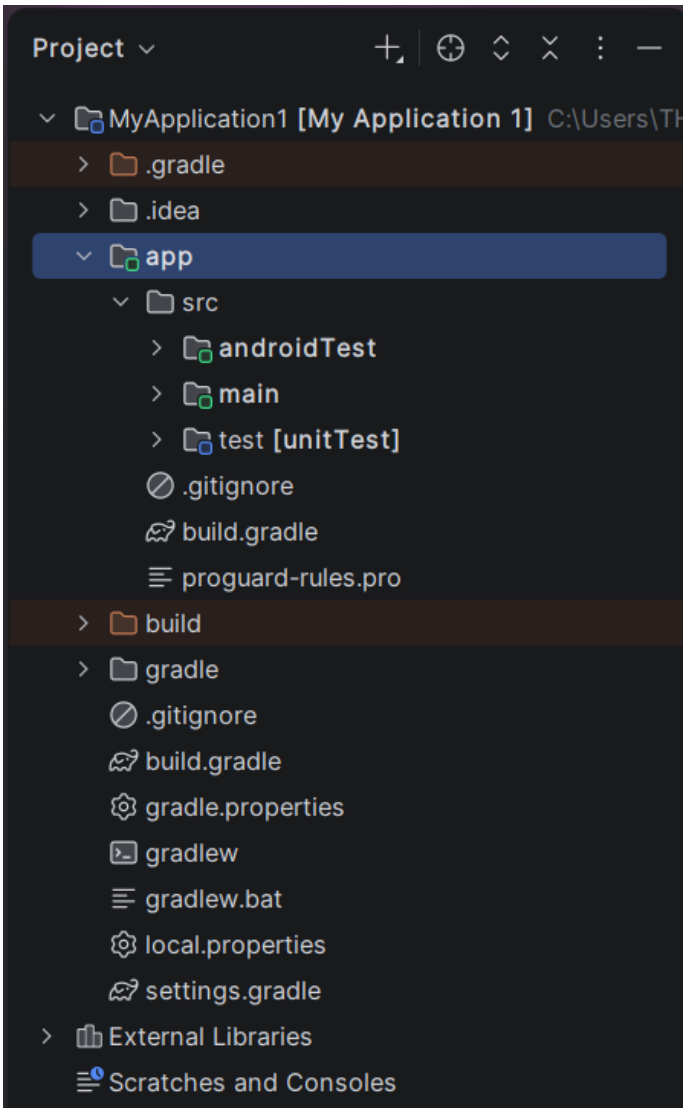
Lab 2: Configuration and System Folders, ListView, RecyclerView, Resource in Android, Important display components, Fragment

I. Configuration and System Folders

Module app

The app module is the central component of an Android project. It contains all source code, resources, and configurations needed to build the application. When running the project, Android Studio builds this module to generate the installation file.

Structure Inside the app Module

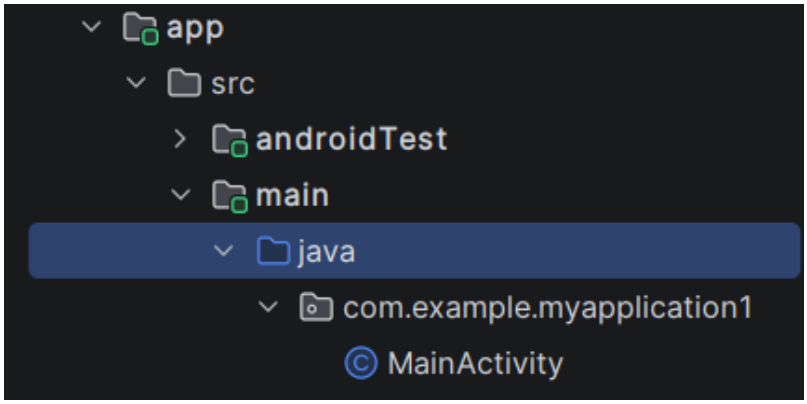


src Folder: The src folder contains all application source code and is divided into three main parts.

main Folder: The main folder is the most important part because it contains all the core content of the application.

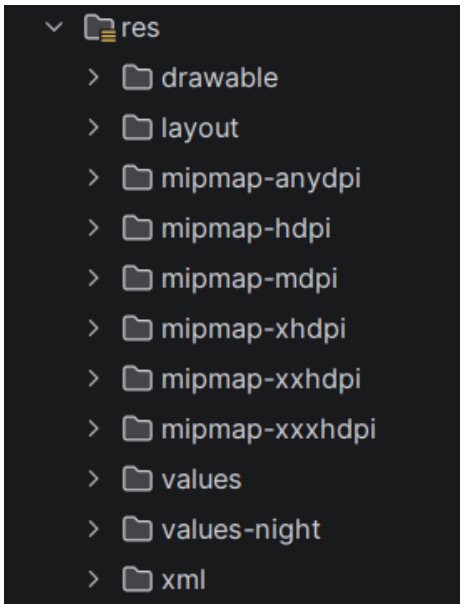
java

The java folder contains the application source code, written in Java or Kotlin. This is where all logic is handled, such as UI control, event handling, API calls, and data management.



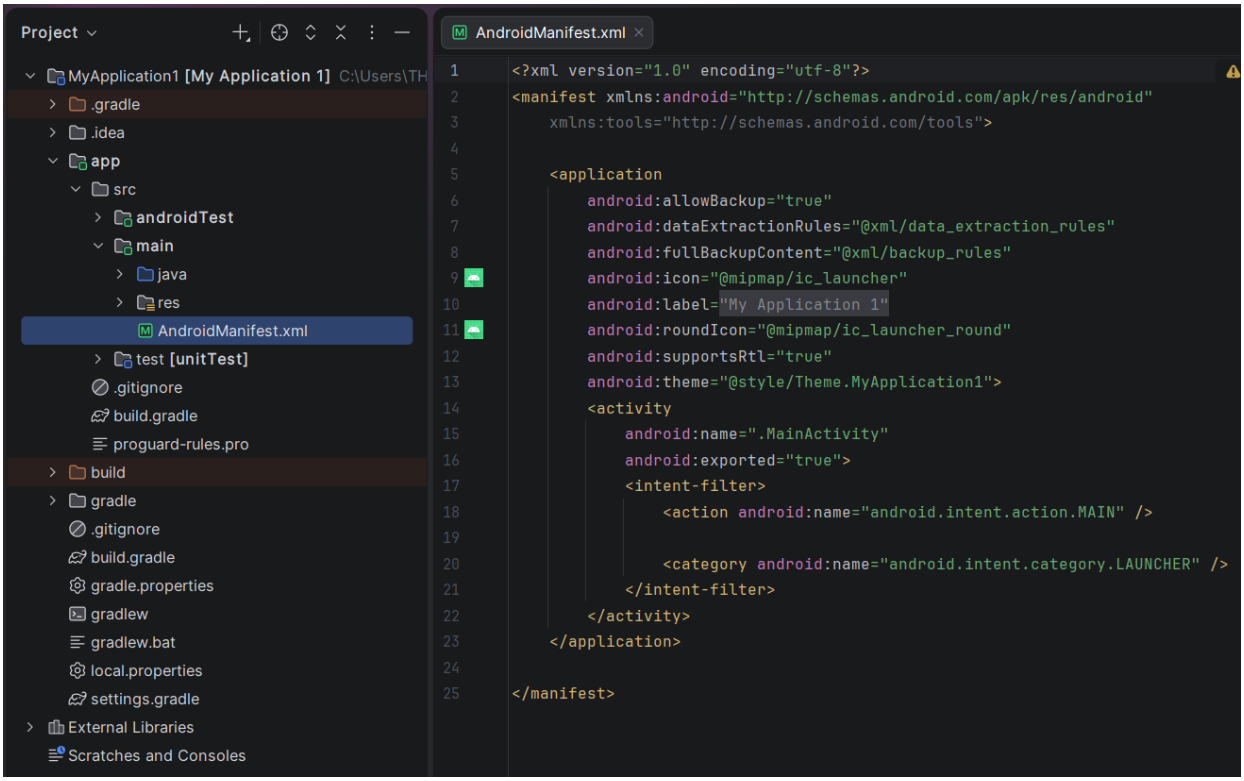
Res

The res folder contains application resources such as XML layouts, images, colors, and string values. Separating resources makes the application easier to maintain and supports multi-language features.



AndroidManifest.xml

The AndroidManifest.xml file declares application information to the Android operating system. It includes components such as activities, permissions, application configuration, and the launch screen.



Gradle

Gradle is an automated build system used in Android to compile and package applications. It acts as an intermediary between source code and the final installation file.

Gradle performs many important tasks during the build process. When a developer runs the application, Gradle reads configuration files, downloads required libraries, compiles source code, processes resources, merges the manifest, and finally generates an APK or AAB file.

Components Related to Gradle

Project-level build.gradle

This file contains general configurations for the entire project, such as plugins and repositories. It manages high-level settings.

Module-level build.gradle

This is the most important file because it defines how the module is built. It includes SDK information, application configuration, and dependency libraries.

settings.gradle

This file defines which modules are included in the project. In a basic project, it usually contains only the app module.

Gradle Workflow

When running an application, Gradle performs a sequence of steps.

First, it reads configuration files to determine how to build.

Next, it downloads dependencies if they are missing.

Then, Java or Kotlin code is compiled. Resources in the res folder are processed and converted into a usable format.

The manifest is merged with other configurations.

Finally, everything is packaged into an APK file and installed on the device.

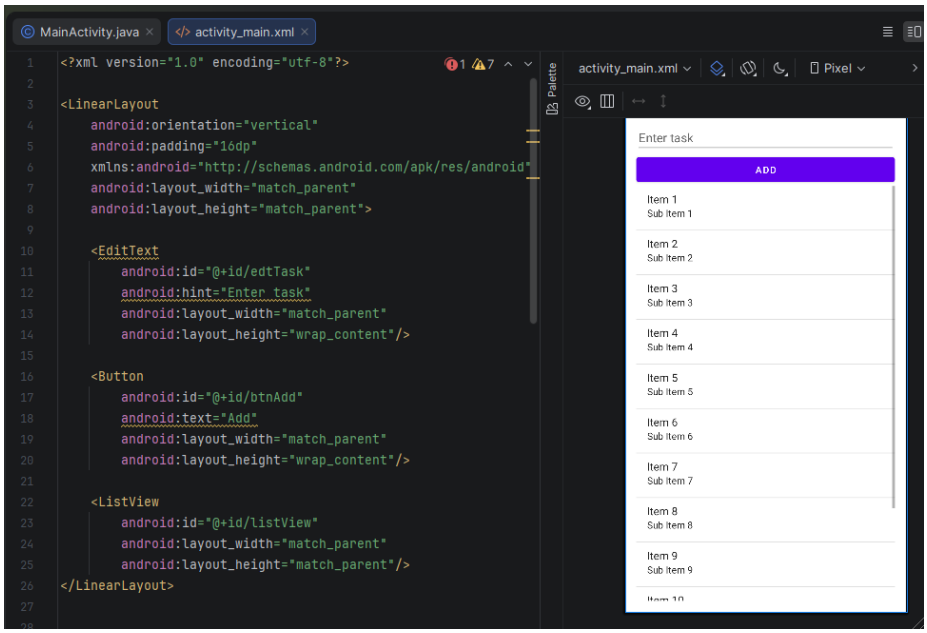
II. ListView, RecyclerView

ListView

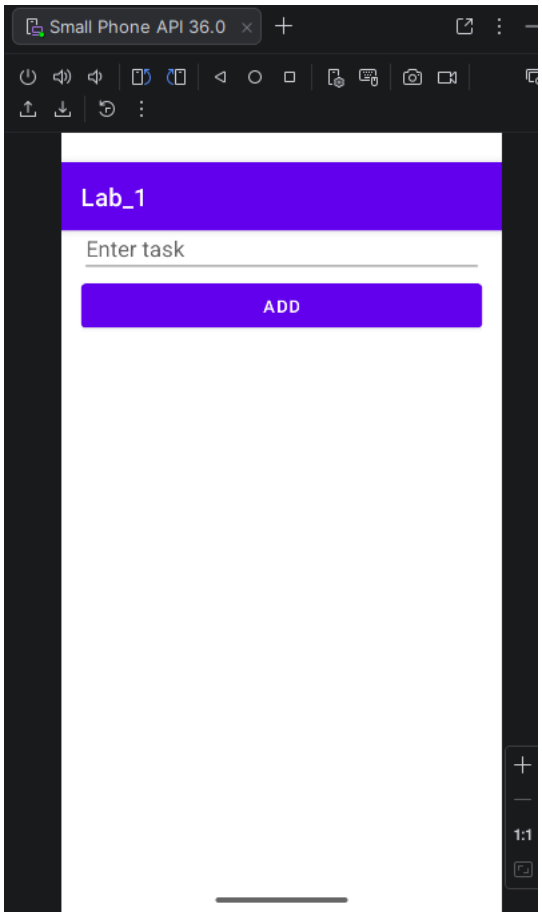
In the previous exercise, we can ListView as a simple approach.

MainActivity.java

```java

public class MainActivity extends AppCompatActivity {

    3 usages
    EditText edtTask;
    2 usages
    Button btnAdd;
    2 usages
    ListView listView;

    3 usages
    ArrayList<String> taskList;
    3 usages
    ArrayAdapter<String> adapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        edtTask = findViewById(R.id.edtTask);
        btnAdd = findViewById(R.id.btnAdd);
        listView = findViewById(R.id.listView);

        taskList = new ArrayList<>();
        adapter = new ArrayAdapter<>( context: this,
                android.R.layout.simple_list_item_1,
                taskList);

        listView.setAdapter(adapter);

        btnAdd.setOnClickListener( View v -> {
            String task = edtTask.getText().toString();
            if (!task.isEmpty()) {
                taskList.add(task);
                adapter.notifyDataSetChanged();
                edtTask.setText("");
            }
        });
    }
```



However, this approach has limitations in terms of performance, scalability, and maintainability, because every time there is a change, the for loop has to iterate from the beginning again to update and re-render the data.



RecyclerView

The first part is **data**. Each task needs its own information, such as the task name and whether it is completed. This is represented by a separate class, here called Task.

The second part is the **list UI**. Since the list contains many tasks with the same display structure, we do not manually create each row. Instead, we use RecyclerView to efficiently display the list of data.

The third part is the **bridge between data and UI**. RecyclerView does not know how to retrieve data or display it. Therefore, an adapter is needed to connect the Task list with each UI item. This adapter is called TaskAdapter.

Solution Structure

MainActivity: Handles user interactions such as entering a task and pressing the Add button

Task: Stores information about each task

TaskAdapter: Displays tasks in RecyclerView and handles item-level interactions such as checking completion or deleting

Code Explanation:

Task Class

title: stores the task name

completed: stores whether the task is completed (boolean)

```
MainActivity.java   Task.java   TaskAdapter.java
package com.example.lab_1;

6 usages
public class Task {

    2 usages
    private String title;
    3 usages
    private boolean completed;
```

The constructor Task(String title) is used when creating a new task. Initially, the task is not completed, so completed = false.

Methods:

getTitle(): returns the task name

isCompleted(): checks completion status

setCompleted(): updates completion status

```
    1 usage
    public Task(String title) {
        this.title = title;
        this.completed = false;
    }

    1 usage
    public String getTitle() {
        return title;
    }

    2 usages
    public boolean isCompleted() {
        return completed;
    }

    1 usage
    public void setCompleted(boolean completed) {
        this.completed = completed;
    }
}
```

TaskAdapter: bridge between DATA and UI

Specifically:

Data: List<Task> taskList

UI: RecyclerView + item_task.xml

Adapter responsibilities:

Get each Task

Bind data to UI

Handle item events (check, delete)

Without an Adapter → RecyclerView cannot display anything.

Functions in TaskAdapter

TaskAdapter(List<Task> taskList)

Purpose: receive task data

Input: List<Task>

Output: none (constructor)

```java
TaskAdapter.java
package com.example.lab_1;

import ...

1 usage
public class TaskAdapter extends RecyclerView.Adapter<TaskAdapter.TaskViewHolder> {

    4 usages
    List<Task> taskList;

    no usages
    public TaskAdapter(List<Task> taskList) { this.taskList = taskList; }

    5 usages
    public class TaskViewHolder extends RecyclerView.ViewHolder {...}

    @Override
    public TaskViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {...}

    @Override
    public void onBindViewHolder(TaskViewHolder holder, int position) {...}

    2 usages
    private void updateUI(TaskViewHolder holder, Task task) {...}

    @Override
    public int getItemCount() { return taskList.size(); }
}
```

Data Source

This is the main data source

Contains all tasks

Adapter only reads from it to display

```java
public class TaskAdapter extends RecyclerView.Adapter<TaskAdapter.TaskViewHolder> {
    4 usages
    List<Task> taskList;

```

Constructor

When creating the adapter, it receives data from MainActivity.

```java
    1 usage
    public TaskAdapter(List<Task> taskList) {
        this.taskList = taskList;
    }

```

ViewHolder

This is the constructor of TaskViewHolder.

It maps UI components in an item:

- txtTask: display task name
- checkTask: mark completion
- btnDelete: delete task

```java
    5 usages
    public class TaskViewHolder extends RecyclerView.ViewHolder {

        4 usages
        TextView txtTask;
        3 usages
        CheckBox checkTask;
        2 usages
        Button btnDelete;

        1 usage
        public TaskViewHolder(View itemView) {
            super(itemView);

            txtTask = itemView.findViewById(R.id.txtTask);
            checkTask = itemView.findViewById(R.id.checkTask);
            btnDelete = itemView.findViewById(R.id.btnDelete);
        }
    }
```

Instead of repeatedly calling findViewById, we map once and reuse → improves performance and cleaner code.

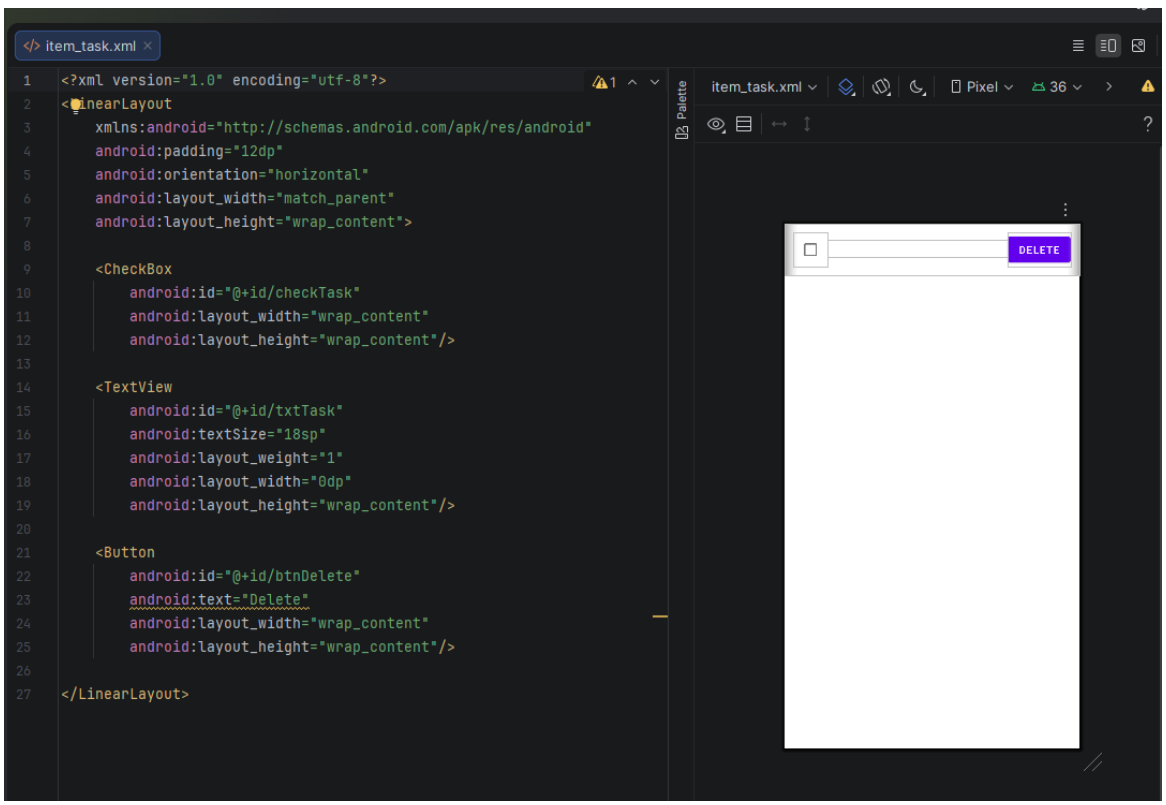
onCreateViewHolder(ViewGroup parent, int viewType)

Creates the UI for each item when RecyclerView needs a new row.

```java
@Override
public TaskViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {

    View view = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.item_task, parent, attachToRoot: false);

    return new TaskViewHolder(view);
}
```

item_task.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:padding="12dp"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <CheckBox
        android:id="@+id/checkTask"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <TextView
        android:id="@+id/txtTask"
        android:textSize="18sp"
        android:layout_weight="1"
        android:layout_width="0dp"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnDelete"
        android:text="Delete"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

</LinearLayout>
```



onBindViewHolder(TaskViewHolder holder, int position)

The most important method in the adapter.

Responsibilities:

- Bind data to UI
- Handle user interactions

Usage:

- Get the task by position
- Set the task name to the TextView
- Set the completed status to the CheckBox
- Update the UI based on the status
- Handle when the user checks the CheckBox
- Handle when the delete button is clicked

```java
@Override
public void onBindViewHolder(TaskViewHolder holder, int position) {

    Task task = taskList.get(position);

    holder.txtTask.setText(task.getTitle());
    holder.checkTask.setChecked(task.isCompleted());

    updateUI(holder, task);

    holder.checkTask.setOnCheckedChangeListener((CompoundButton buttonView, boolean isChecked) -> {
        task.setCompleted(isChecked);
        updateUI(holder, task);
    });

    holder.btnDelete.setOnClickListener(View v -> {
        taskList.remove(position);
        notifyDataSetChanged();
    });
}
```

updateUI(TaskViewHolder holder, Task task)

update UI based on task status

If completed:

- Text color: gray
- Background: light green

If not completed:

- Text color: black
- Background: white

```
2 usages
private void updateUI(TaskViewHolder holder, Task task) {

    if (task.isCompleted()) {

        holder.txtTask.setTextColor(Color.GRAY);
        holder.itemView.setBackgroundColor(Color.parseColor( colorString: "#DFF0D8"));

    } else {

        holder.txtTask.setTextColor(Color.BLACK);
        holder.itemView.setBackgroundColor(Color.WHITE);

    }
}

@Override
public int getItemCount() {
    return taskList.size();
}
```

getItemCount(): Returns number of items

Hàm này cho RecyclerView biết có bao nhiêu item cần hiển thị.

notifyDataSetChanged(): Notifies RecyclerView that data has changed and UI needs to be refreshed

MainActivity

```
public class MainActivity extends AppCompatActivity
```

AppCompatActivity is a class provided by Android that helps:

Manage Activity Lifecycle

- onCreate
- onStart
- onResume
- onPause
- onDestroy

```
TaskAdapter.java    MainActivity.java    AppCompatActivity.java ×
82      public class AppCompatActivity extends FragmentActivity implements AppCompatCallback,
243         @Override
244         protected void onPostResume() {
245             super.onPostResume();
246             getDelegate().onPostResume();
247         }
248
249         @Override
250         protected void onStart() {
251             super.onStart();
252             getDelegate().onStart();
253         }
254
255         @Override
256         protected void onStop() {
257             super.onStop();
258             getDelegate().onStop();
259         }
```

UI Components

- edtTask: input field
- btnAdd: add button
- recyclerTasks: display list

```
public class MainActivity extends AppCompatActivity {

    3 usages
    EditText edtTask;
    2 usages
    Button btnAdd;
    3 usages
    RecyclerView recyclerTasks;
```

Data + Adapter

- taskList: stores tasks
- adapter: connects data and UI

```
3 usages
ArrayList<Task> taskList;
3 usages
TaskAdapter adapter;
```

Code for handling

```
MainActivity.java
public class MainActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {
        edtTask = findViewById(R.id.edtTask);
        btnAdd = findViewById(R.id.btnAdd);
        recyclerTasks = findViewById(R.id.recyclerTasks);

        taskList = new ArrayList<>();

        adapter = new TaskAdapter(taskList);

        recyclerTasks.setLayoutManager(new LinearLayoutManager( context: this));
        recyclerTasks.setAdapter(adapter);

        btnAdd.setOnClickListener( View v -> {

            String taskName = edtTask.getText().toString();

            if (!taskName.isEmpty()) {

                taskList.add(new Task(taskName));
                adapter.notifyDataSetChanged();

                edtTask.setText("");

            } else {

                Toast.makeText( context: this,  text: "Enter task first", Toast.LENGTH_SHORT).show();

            }

        });
    }
```

Declare the views for mapping

```java
MainActivity.java

public class MainActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {
        edtTask = findViewById(R.id.edtTask);
        btnAdd = findViewById(R.id.btnAdd);
        recyclerTasks = findViewById(R.id.recyclerTasks);
```

Initialize the task list

```java

        taskList = new ArrayList<>();
```

Initialize the adapter for tasks

```java

        adapter = new TaskAdapter(taskList);

        recyclerTasks.setLayoutManager(new LinearLayoutManager( context: this));
        recyclerTasks.setAdapter(adapter);
```

Handle events

```java

        btnAdd.setOnClickListener( View v -> {

            String taskName = edtTask.getText().toString();

            if (!taskName.isEmpty()) {

                taskList.add(new Task(taskName));
                adapter.notifyDataSetChanged();

                edtTask.setText("");

            } else {

                Toast.makeText( context: this,  text: "Enter task first", Toast.LENGTH_SHORT).show();

            }

        });
    }
```

main_activity.xml:

MainActivity.java | activity_main.xml | Task.java

```
<LinearLayout
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:layout_marginTop="12dp">

        <EditText
            android:id="@+id/edtTask"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:hint="Enter new task"
            android:padding="10dp"/>

        <Button
            android:id="@+id/btnAdd"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="ADD"
            android:backgroundTint="#6200EE"
            android:textColor="#FFFFFF"
            android:layout_marginStart="8dp"/>
    </LinearLayout>
```

```
            android:textColor="#FFFFFF"
            android:layout_marginStart="8dp"/>
    </LinearLayout>

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerTasks"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_marginTop="12dp"
        android:layout_weight="1"/>
```

III. Resource in Android

In Android, the res folder is used to store resources that are not Java code, such as images, colors, text strings, XML layouts, animations, and menus. Android automatically generates the R class so that we can access these resources in code.

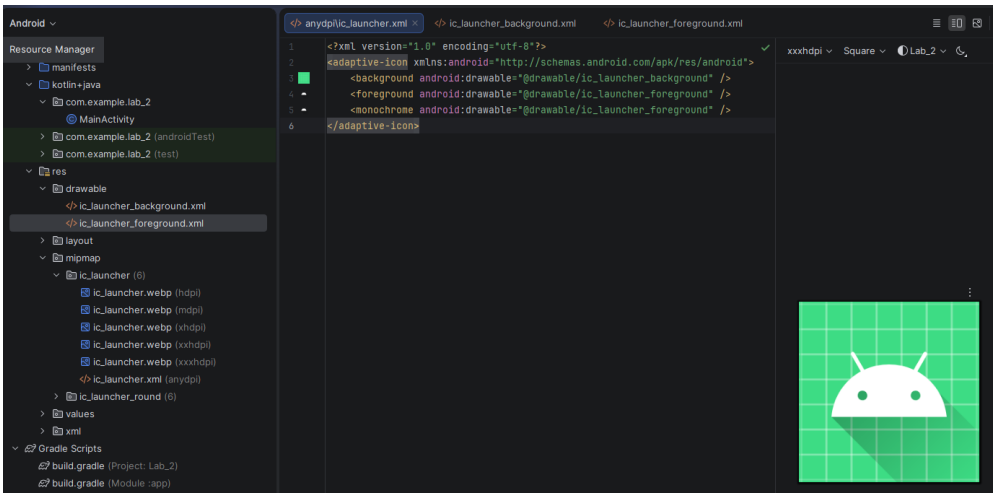
1. mipmap

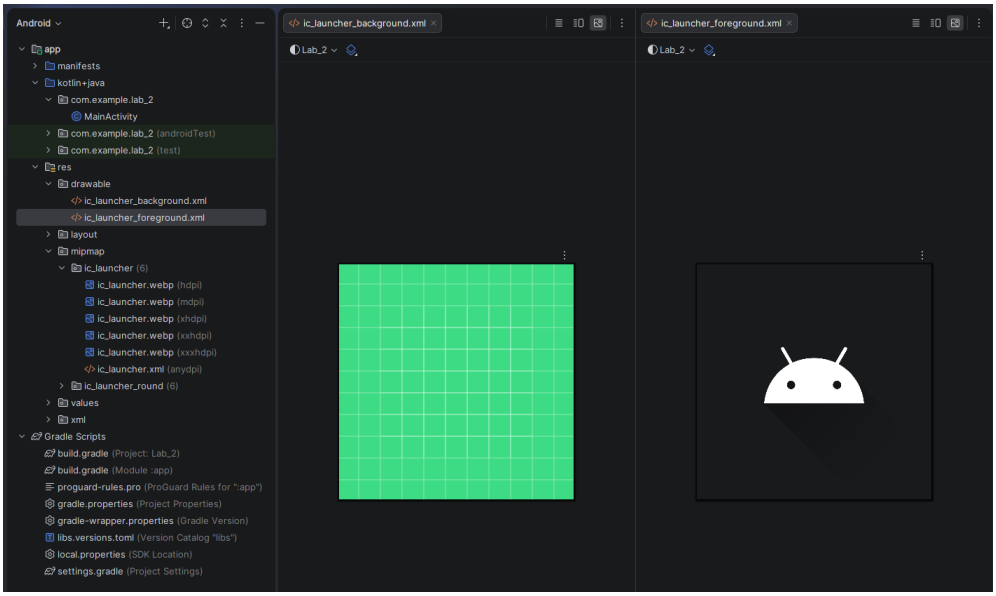
mipmap is usually used to store the launcher icon of the application.

Example:

android:icon="@mipmap/ic_launcher"

When the app is installed on devices with different screen resolutions, Android prefers using mipmap for the main application icon.





2. drawable

drawable contains images and other graphic resources such as:

- local images
- local icons
- layer-list
- selector
- clip
- shape

a. Regular image

Place the image file in res/drawable, for example logo.png.

Use in XML:

```
<ImageView
   android:layout_width="120dp"
   android:layout_height="120dp"
   android:src="@drawable/logo" />
```

Use in Java:

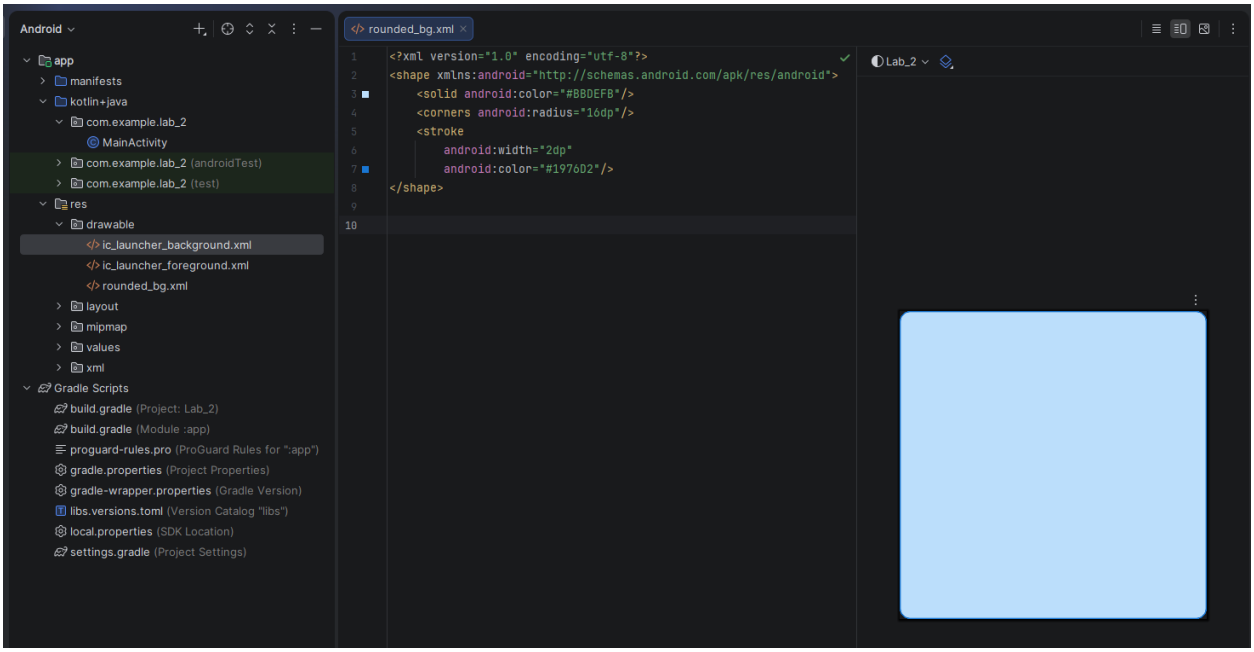
```
ImageView img = findViewById(R.id.imgLogo);
img.setImageResource(R.drawable.logo);
```

b. Shape

Used to create a background with XML instead of using an image.

Example res/drawable/rounded_bg.xml

```
<shape xmlns:android="http://schemas.android.com/apk/res/android">
   <solid android:color="#BBDEFB"/>
   <corners android:radius="16dp"/>
   <stroke
      android:width="2dp"
      android:color="#1976D2"/>
</shape>
```



Usage:

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello"
    android:padding="12dp"
    android:background="@drawable/rounded_bg"/>
```

c. Selector

Used to change the appearance depending on state, for example whether a button is pressed or not.

Example res/drawable/button_selector.xml

```
<selector xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:state_pressed="true">
        <shape android:shape="rectangle">
            <solid android:color="#FF0000"/>
            <corners android:radius="16dp"/>
        </shape>
    </item>
    <item>
        <shape android:shape="rectangle">
```

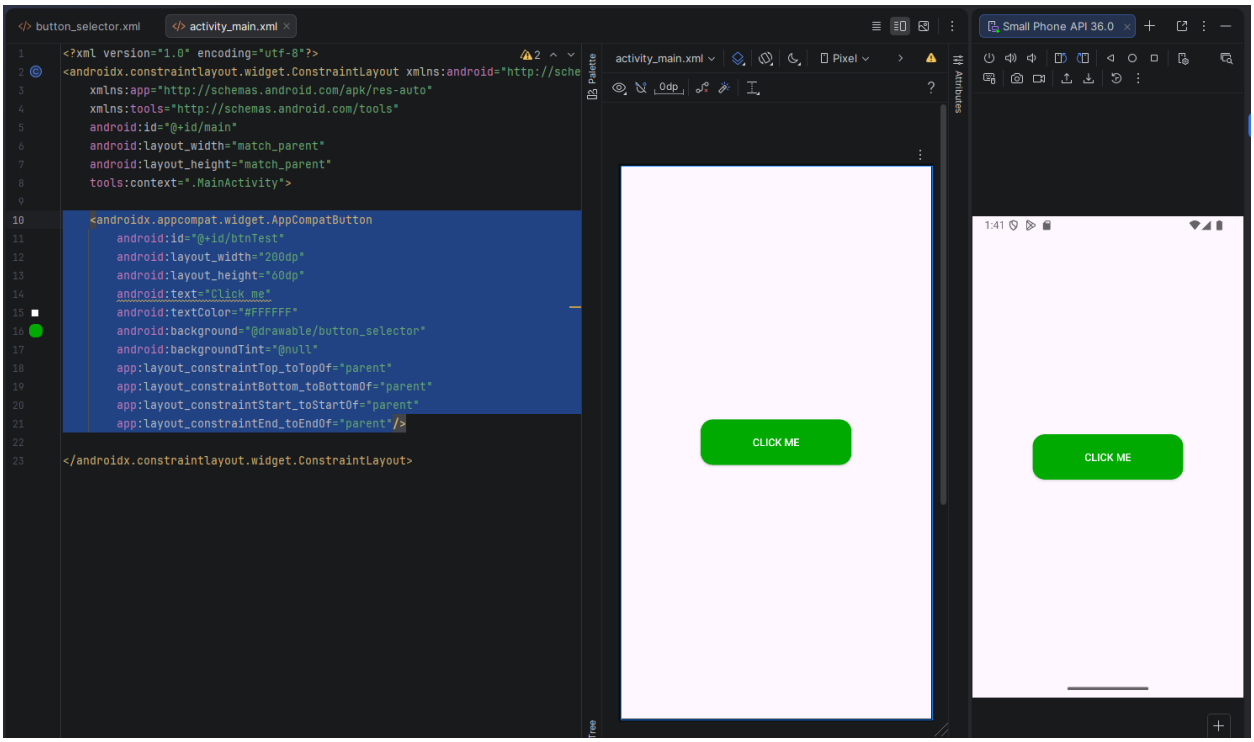
```
        <solid android:color="#00AA00"/>
        <corners android:radius="16dp"/>
      </shape>
    </item>
</selector>
```

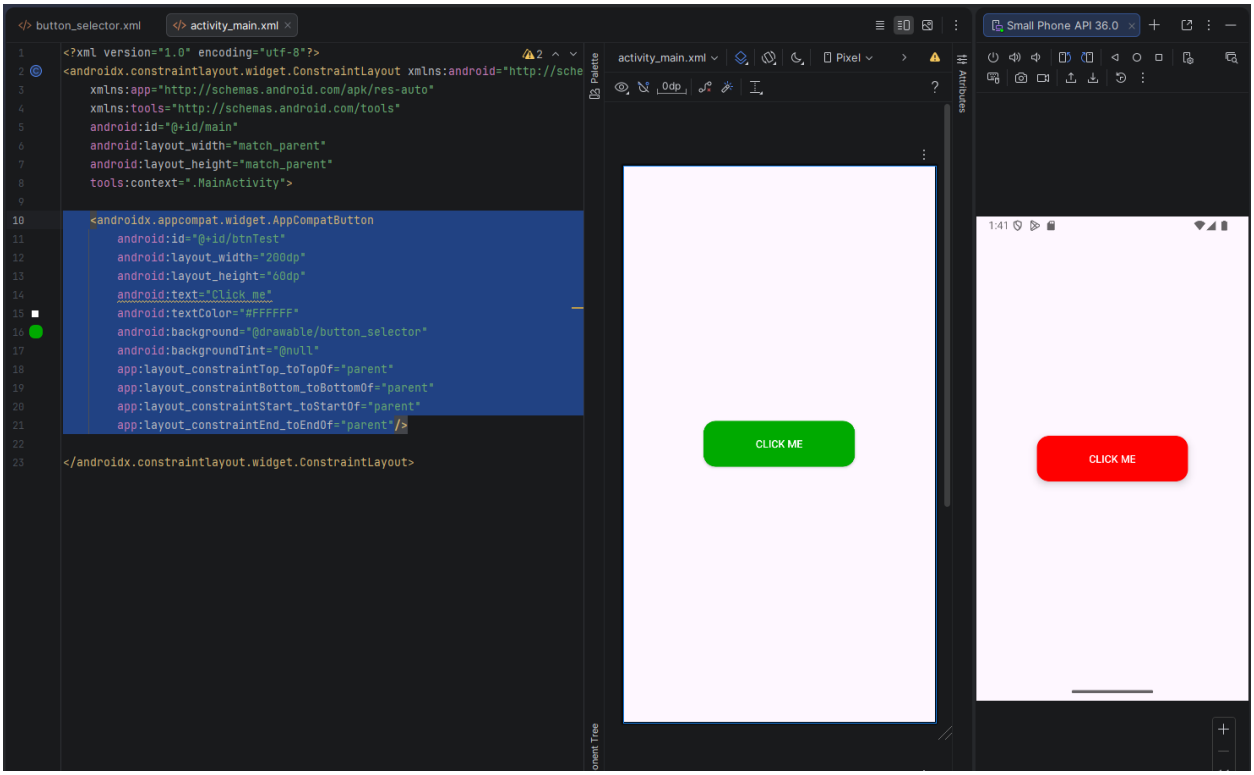
File res/values/colors.xml:

```
button_selector.xml   colors.xml
1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <color name="black">#FF000000</color>
4      <color name="white">#FFFFFFFF</color>
5      <color name="purple_200">#BB86FC</color>
6      <color name="teal_200">#03DAC5</color>
7  </resources>
```

Usage:

```
<androidx.appcompat.widget.AppCompatButton
    android:id="@+id/btnTest"
    android:layout_width="200dp"
    android:layout_height="60dp"
    android:text="Click me"
    android:textColor="#FFFFFF"
    android:background="@drawable/button_selector"
    android:backgroundTint="@null"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"/>
```


button_selector.xml
activity_main.xml
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://sche
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">
<androidx.appcompat.widget.AppCompatButton
android:id="@+id/btnTest"
android:layout_width="200dp"
android:layout_height="60dp"
android:text="Click me"
android:textColor="#FFFFFF"
android:background="@drawable/button_selector"
android:backgroundTint="@null"
app:layout_constraintTop_toTopOf="parent"
app:layout_constraintBottom_toBottomOf="parent"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toEndOf="parent"/>
</androidx.constraintlayout.widget.ConstraintLayout>
Small Phone API 36.0
CLICK ME
CLICK ME

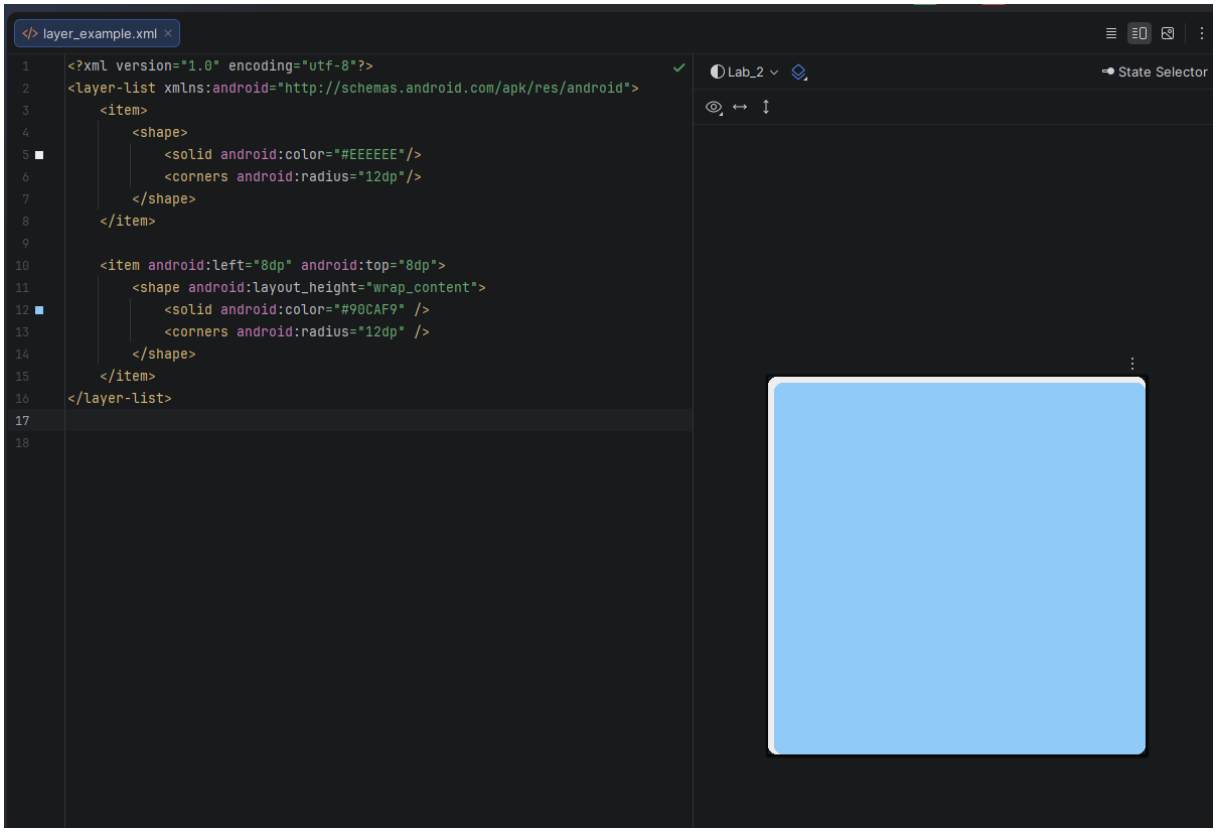

button_selector.xml
activity_main.xml
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://sche
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">
<androidx.appcompat.widget.AppCompatButton
android:id="@+id/btnTest"
android:layout_width="200dp"
android:layout_height="60dp"
android:text="Click me"
android:textColor="#FFFFFF"
android:background="@drawable/button_selector"
android:backgroundTint="@null"
app:layout_constraintTop_toTopOf="parent"
app:layout_constraintBottom_toBottomOf="parent"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toEndOf="parent"/>
</androidx.constraintlayout.widget.ConstraintLayout>
Small Phone API 36.0
CLICK ME
CLICK ME

d. Layer-list

Used to stack multiple drawable layers on top of each other.

Example res/drawable/layer_example.xml

```
<layer-list xmlns:android="http://schemas.android.com/apk/res/android">
    <item>
        <shape>
            <solid android:color="#EEEEEE"/>
            <corners android:radius="12dp"/>
        </shape>
    </item>
    <item android:left="8dp" android:top="8dp">
        <shape>
            <solid android:color="#90CAF9"/>
            <corners android:radius="12dp"/>
        </shape>
    </item>
</layer-list>
```



3. values

The values folder usually contains:

- strings.xml
- colors.xml
- dimens.xml
- styles.xml
- themes.xml

a. strings.xml

```
<resources>
   <string name="app_name">MyApp</string>
   <string name="hello">Xin chào Android</string>
</resources>
```

Usage:

```
android:text="@string/hello"
```

b. colors.xml

```
<resources>
   <color name="primary">#6200EE</color>
   <color name="gray">#888888</color>
</resources>
```

Usage:

```
android:textColor="@color/gray"
```

c. dimens.xml

```
<resources>
   <dimen name="padding_large">16dp</dimen>
   <dimen name="text_big">20sp</dimen>
</resources>
```

Usage:

```
android:padding="@dimen/padding_large"
android:textSize="@dimen/text_big"
```

d. styles và themes

Used to define reusable UI styles.

Example styles.xml

```
<resources>
   <style name="MyTextStyle">
      <item name="android:textSize">18sp</item>
      <item name="android:textColor">#0000FF</item>
   </style>
</resources>
```

Usage:

```
<TextView
   style="@style/MyTextStyle"
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:text="Styled Text"/>
```

4. values-night

Used to support Dark Mode.

Example:

- res/values/colors.xml
- res/values-night/colors.xml

Day mode: <color name="background">#FFFFFF</color>

Night mode: <color name="background">#121212</color>

Android automatically chooses the appropriate version depending on light or dark mode.

5. values-country code

Used to create resources for a specific language or country.

Example:

- values/strings.xml
- values-vi/strings.xml

- values-en/strings.xml

values-vi/strings.xml:

```
<resources>
   <string name="greeting">Xin chào</string>
</resources>
```

values-en/strings.xml:

```
<resources>
   <string name="greeting">Hello</string>
</resources>
```

6. anim

The res/anim folder contains animation XML files.

Create the res/anim folder

In Android Studio:

- Right click res → New → Android Resource Directory
- Choose resource type → anim

Create animation files

1. Alpha

res/anim/fade_in.xml

```
<?xml version="1.0" encoding="utf-8"?>
<alpha xmlns:android="http://schemas.android.com/apk/res/android"
   android:fromAlpha="0.0"
   android:toAlpha="1.0"
   android:duration="1000"/>
```

Effect: image fades from transparent to visible

2. Scale

res/anim/scale.xml

```
<?xml version="1.0" encoding="utf-8"?>
<scale xmlns:android="http://schemas.android.com/apk/res/android"
   android:fromXScale="0.5"
```

```
    android:toXScale="1.0"
    android:fromYScale="0.5"
    android:toYScale="1.0"
    android:duration="800"
    android:pivotX="50%"
    android:pivotY="50%"/>
```

Effect: grows from small to large

3. Rotate

res/anim/rotate.xml

```
<?xml version="1.0" encoding="utf-8"?>
<rotate xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromDegrees="0"
    android:toDegrees="360"
    android:duration="1000"
    android:pivotX="50%"
    android:pivotY="50%"/>
```

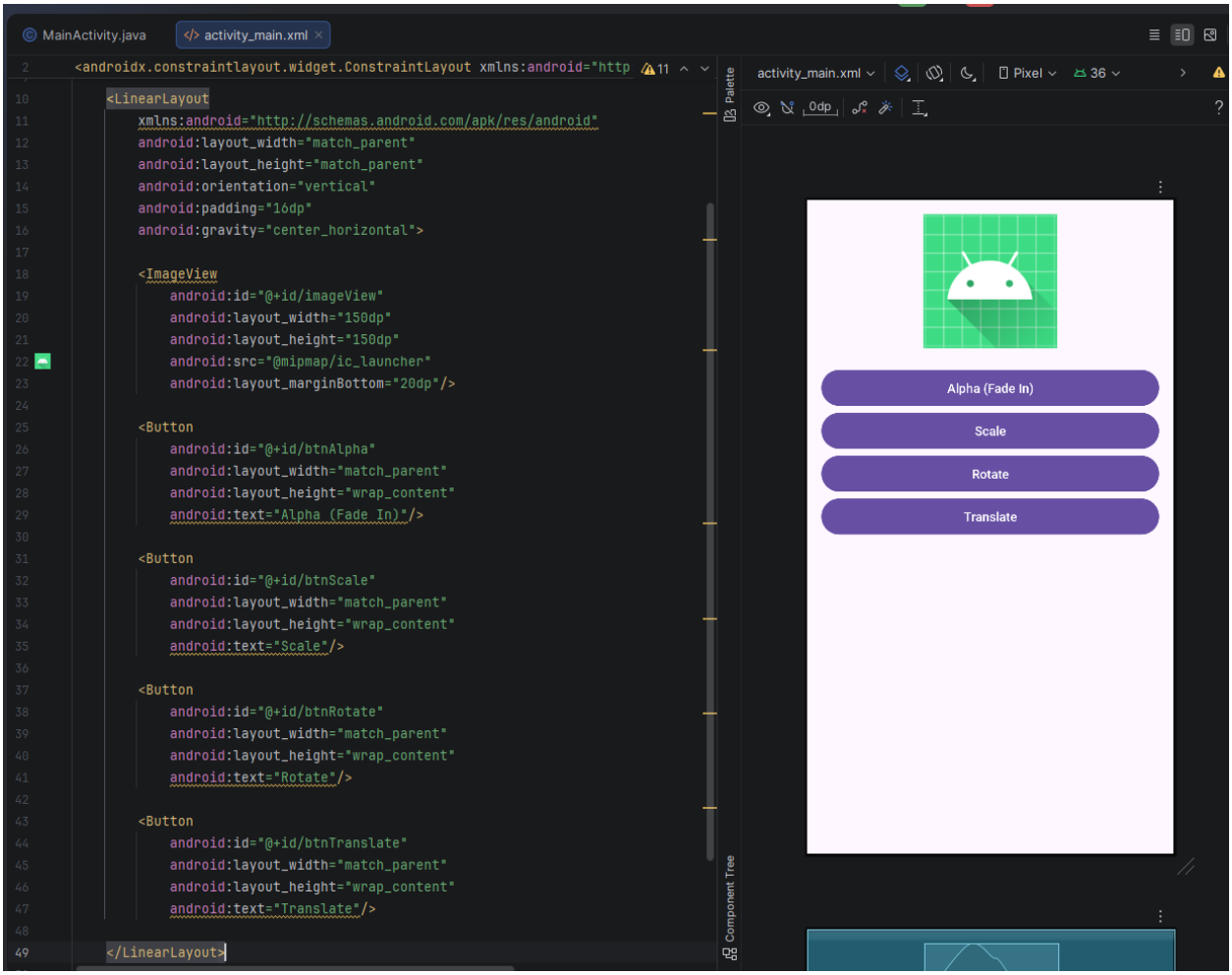
Effect: rotates one full circle

4. Translate

res/anim/translate.xml

```
<?xml version="1.0" encoding="utf-8"?>
<translate xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromXDelta="0"
    android:toXDelta="300"
    android:duration="1000"/>
```

Effect: moves to the right



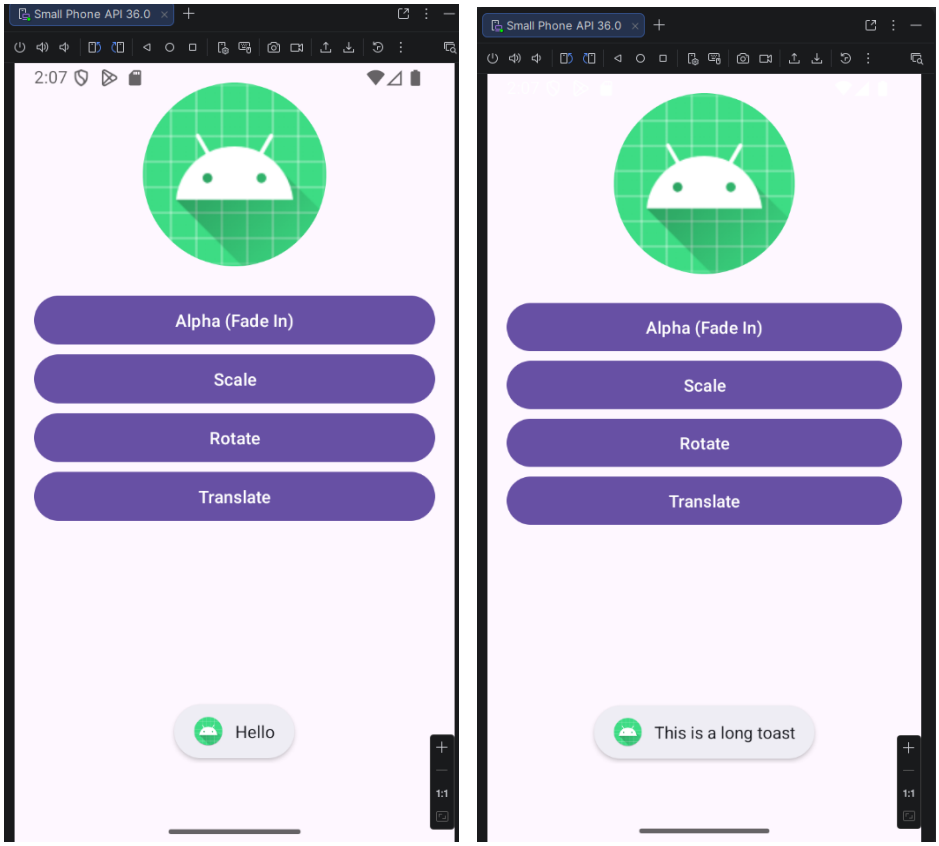
IV. Some other important display components

1. Toast

Toast is a small temporary message.

Short Toast: Toast.makeText(this, "Hello", Toast.LENGTH_SHORT).show();

Long Toast: Toast.makeText(this, "This is a long toast",Toast.LENGTH_LONG).show();



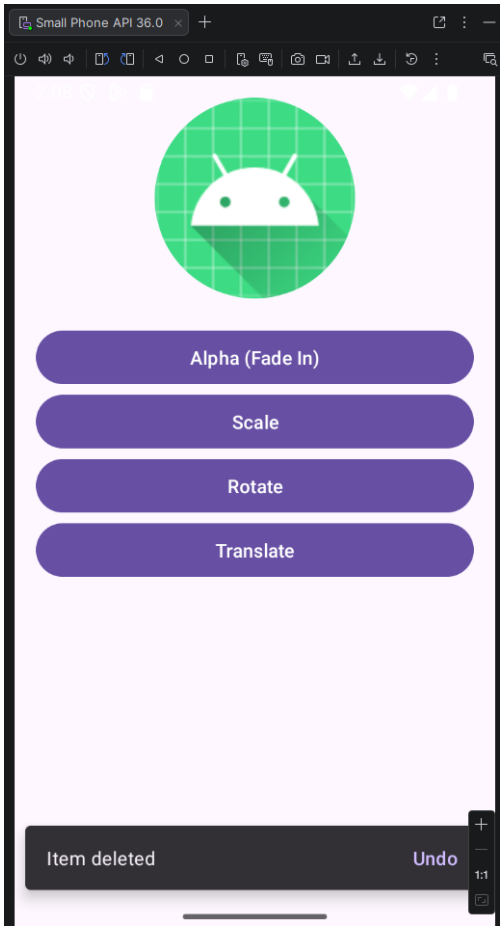
2. Snackbar

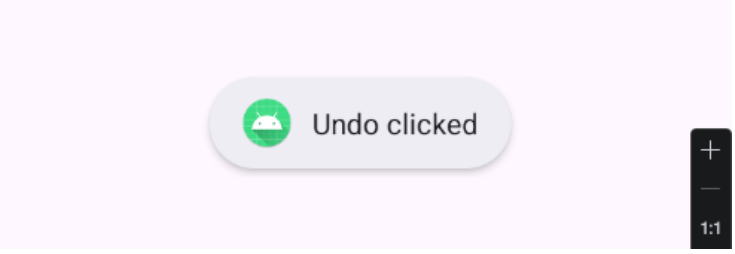
Snackbar is similar to Toast, but it appears at the bottom of the screen and can have an action button.

```
View root = findViewById(android.R.id.content);
Snackbar.make(root, "Item deleted", Snackbar.LENGTH_LONG)
        .setAction("Undo", v -> {
            Toast.makeText(this, "Undo clicked", Toast.LENGTH_SHORT).show();
        })
        .show();
```

```
View root = findViewById(android.R.id.content);
Snackbar.make(root, text: "Item deleted", Snackbar.LENGTH_LONG)
        .setAction( text: "Undo", View v -> {
            Toast.makeText( context: this, text: "Undo clicked", Toast.LENGTH_SHORT).show();
        })
        .show();
```

A custom Snackbar is usually created by customizing the view inside it.





3. Dialog

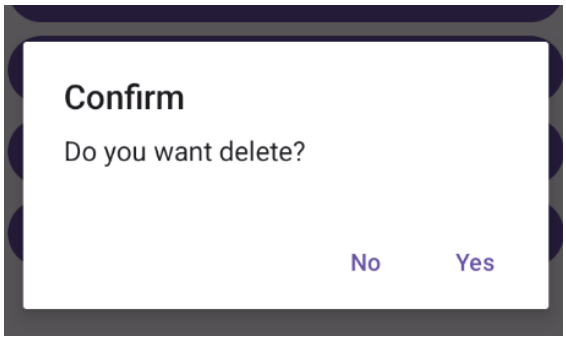
Dialog is a popup window used to notify the user or ask them to make a choice.

Simple AlertDialog

```
new AlertDialog.Builder(this)
    .setTitle("Confirmation")
    .setMessage("Do you want to delete it?")
    .setPositiveButton("Yes", (dialog, which) -> {
    Toast.makeText(this, "Deleted", Toast.LENGTH_SHORT).show();
})
```

.setNegativeButton("No", null)
.show();

```
new AlertDialog.Builder( context: this)
        .setTitle("Confirmation")
        .setMessage("Do you want to delete it?")
        .setPositiveButton( text: "Yes", ( DialogInterface dialog,  int which) -> {
            Toast.makeText( context: this,  text: "Deleted", Toast.LENGTH_SHORT).show();
        })
        .setNegativeButton( text: "No",  listener: null)
        .show();
```

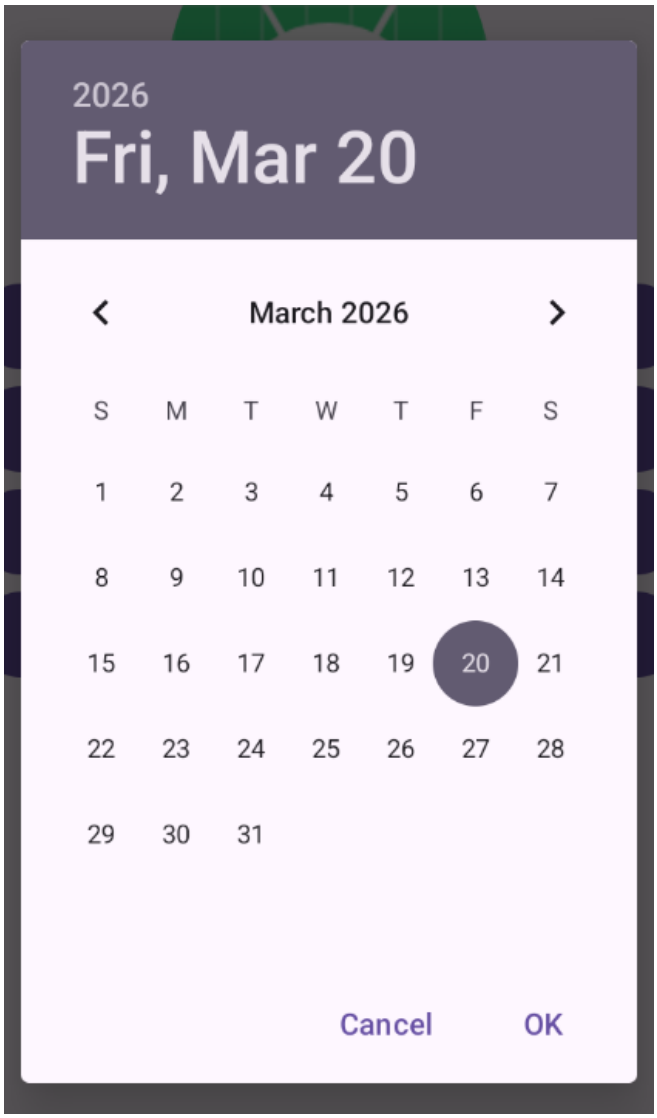


6. DatePickerDialog và TimePickerDialog

DatePickerDialog

Calendar calendar = Calendar.getInstance();
DatePickerDialog dialog = new DatePickerDialog(this,
    (view, year, month, dayOfMonth) -> {
        Toast.makeText(this, dayOfMonth + "/" + (month + 1) + "/" + year,
Toast.LENGTH_SHORT).show();
    },
    calendar.get(Calendar.YEAR),
    calendar.get(Calendar.MONTH),
    calendar.get(Calendar.DAY_OF_MONTH));
dialog.show();

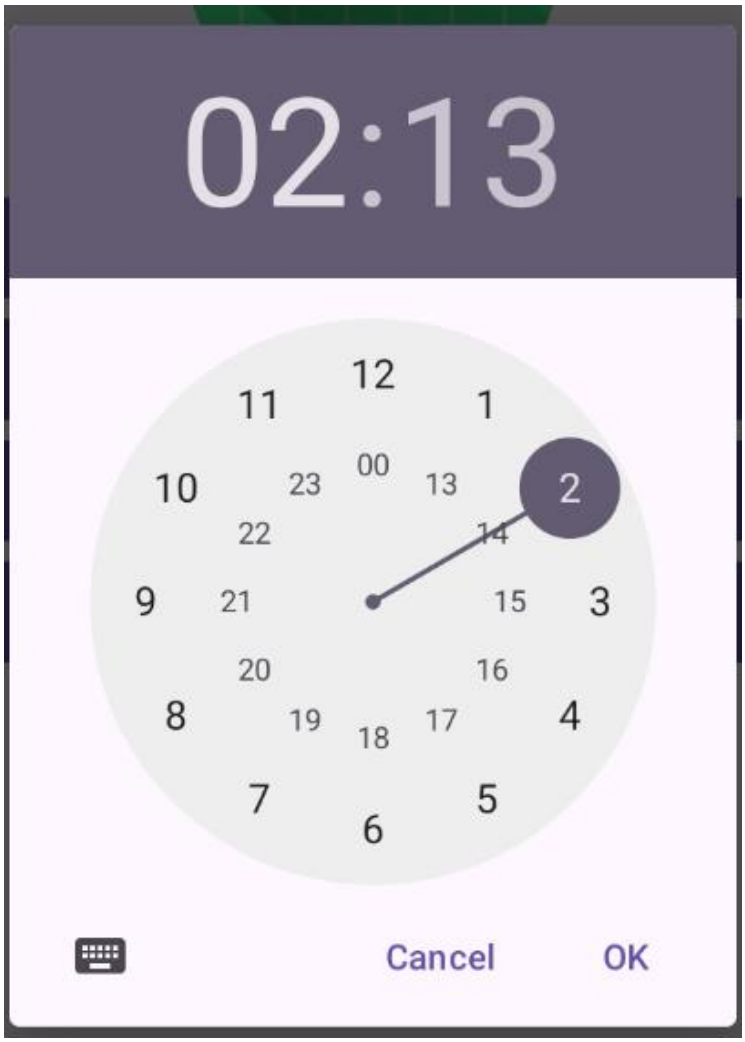
```
Calendar calendar = Calendar.getInstance();
DatePickerDialog dialog = new DatePickerDialog( context: this,
        ( DatePicker view,  int year,  int month,  int dayOfMonth) -> {
            Toast.makeText( context: this,  text: dayOfMonth + "/" + (month + 1) + "/" + year, Toast.LENGTH_SHORT).show();
        },
        calendar.get(Calendar.YEAR),
        calendar.get(Calendar.MONTH),
        calendar.get(Calendar.DAY_OF_MONTH));
dialog.show();
```



TimePickerDialog

```
Calendar calendar = Calendar.getInstance();
TimePickerDialog timeDialog = new TimePickerDialog(this,
        (view, hourOfDay, minute) -> {
            Toast.makeText(this, hourOfDay + ":" + minute,
Toast.LENGTH_SHORT).show();
        },
        calendar.get(Calendar.HOUR_OF_DAY),
        calendar.get(Calendar.MINUTE),
        true);
timeDialog.show();
```

```
TimePickerDialog timeDialog = new TimePickerDialog( context: this,
        ( TimePicker view,  int hourOfDay,  int minute) -> {
            Toast.makeText( context: this,  text: hourOfDay + ":" + minute, Toast.LENGTH_SHORT).show();
        },
        calendar.get(Calendar.HOUR_OF_DAY),
        calendar.get(Calendar.MINUTE),
         is24HourView: true);
timeDialog.show();
```



V. Fragment

1. What is a Fragment

A Fragment is a reusable part of an Activity's user interface and logic.

If an Activity is considered a complete screen, then a Fragment is a smaller part inside that screen. One Activity can contain one or many Fragments.

Fragments are useful because they help:

- split the UI into smaller parts
- reuse UI in different screens
- manage code more easily
- support flexible layouts for phones and tablets

For example:

- a shopping app may have a ProductListFragment and a ProductDetailFragment
- a news app may have HomeFragment, FavoriteFragment, and ProfileFragment

A Fragment cannot exist alone. It must be attached to an Activity.

2. Fragment Lifecycle

A Fragment has its own lifecycle. The most common lifecycle methods are:

- onAttach(): The Fragment is attached to the Activity.
- onCreate(): The Fragment is created.
- onCreateView(): The Fragment creates its UI and returns the layout.
- onViewCreated(): Called after the view has been created.
- onStart(): The Fragment becomes visible.
- onResume(): The Fragment becomes active and the user can interact with it.
- onPause(): The Fragment is partially hidden.
- onStop(): The Fragment is no longer visible.
- onDestroyView(): The Fragment's view is removed.
- onDestroy(): The Fragment is destroyed.

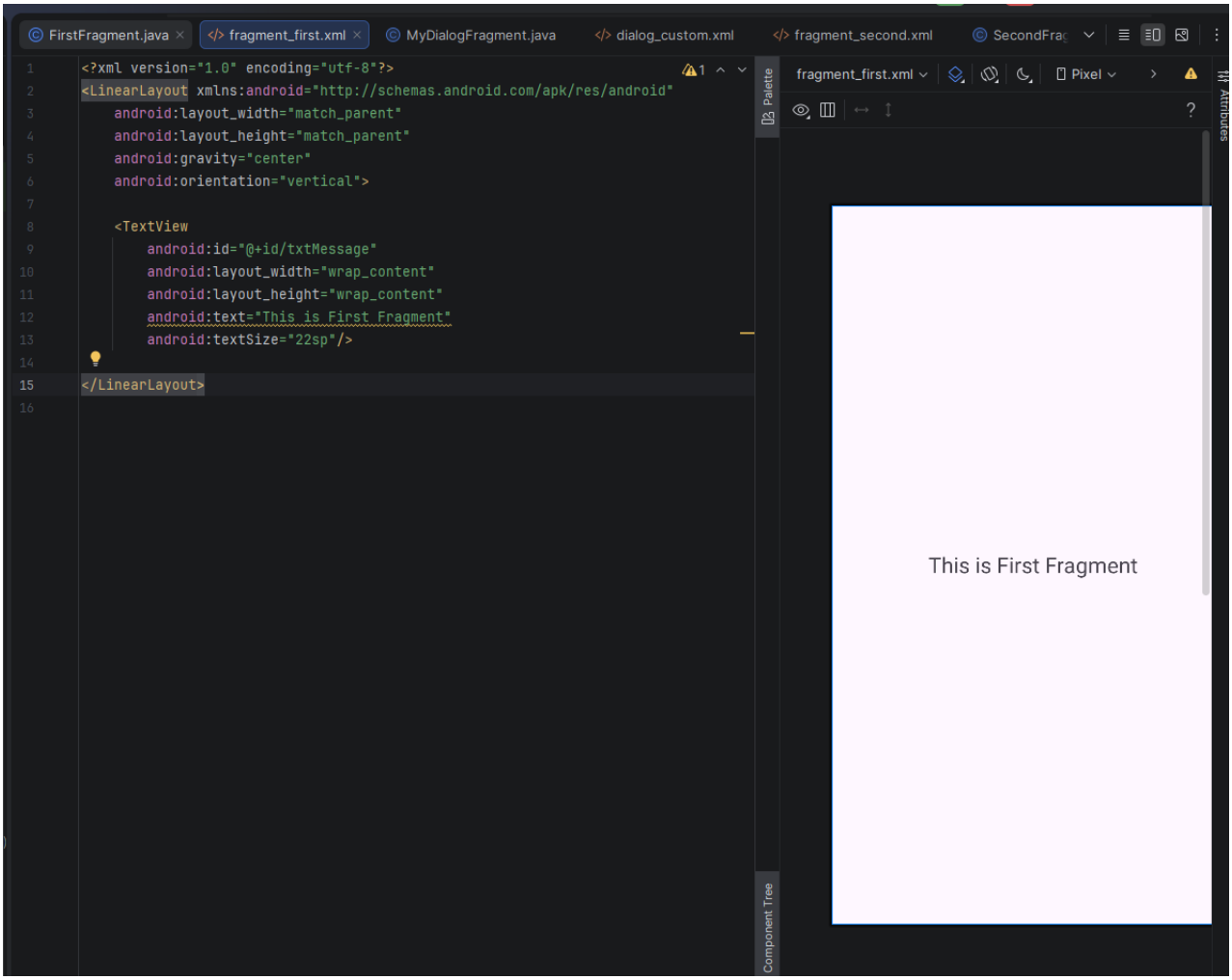
- onDetach(): The Fragment is detached from the Activity.

3. Create a Fragment

To create a Fragment, you usually need:

- a Java class that extends Fragment
- an XML layout for that Fragment

3.1 Fragment layout: fragment_first.xml



3.2 Java class: FirstFragment.java

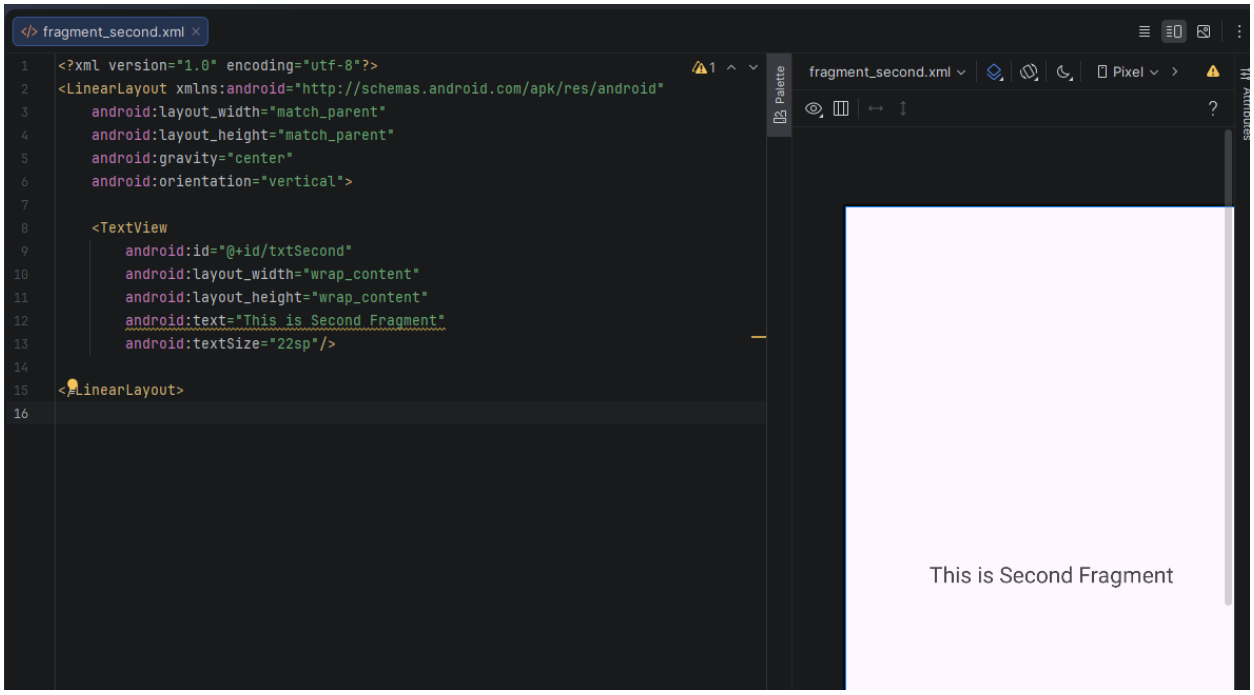
```java
import androidx.fragment.app.Fragment;

public class FirstFragment extends Fragment {

    2 usages
    private TextView txtMessage;

    public FirstFragment() {
    }

    1 usage
    public static FirstFragment newInstance(String message) {
        FirstFragment fragment = new FirstFragment();
        Bundle args = new Bundle();
        args.putString("message_key", message);
        fragment.setArguments(args);
        return fragment;
    }

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_first, container, attachToRoot: false);

        txtMessage = view.findViewById(R.id.txtMessage);

        if (getArguments() != null) {
            String message = getArguments().getString( key: "message_key");
            if (!TextUtils.isEmpty(message)) {
                txtMessage.setText(message);
            }
        }

        return view;
    }
}
```

This Fragment uses a newInstance() method to receive data from the Activity.

4. Activity with Multiple Fragments

fragment_second.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <TextView
        android:id="@+id/txtSecond"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is Second Fragment"
        android:textSize="22sp"/>

</LinearLayout>
```



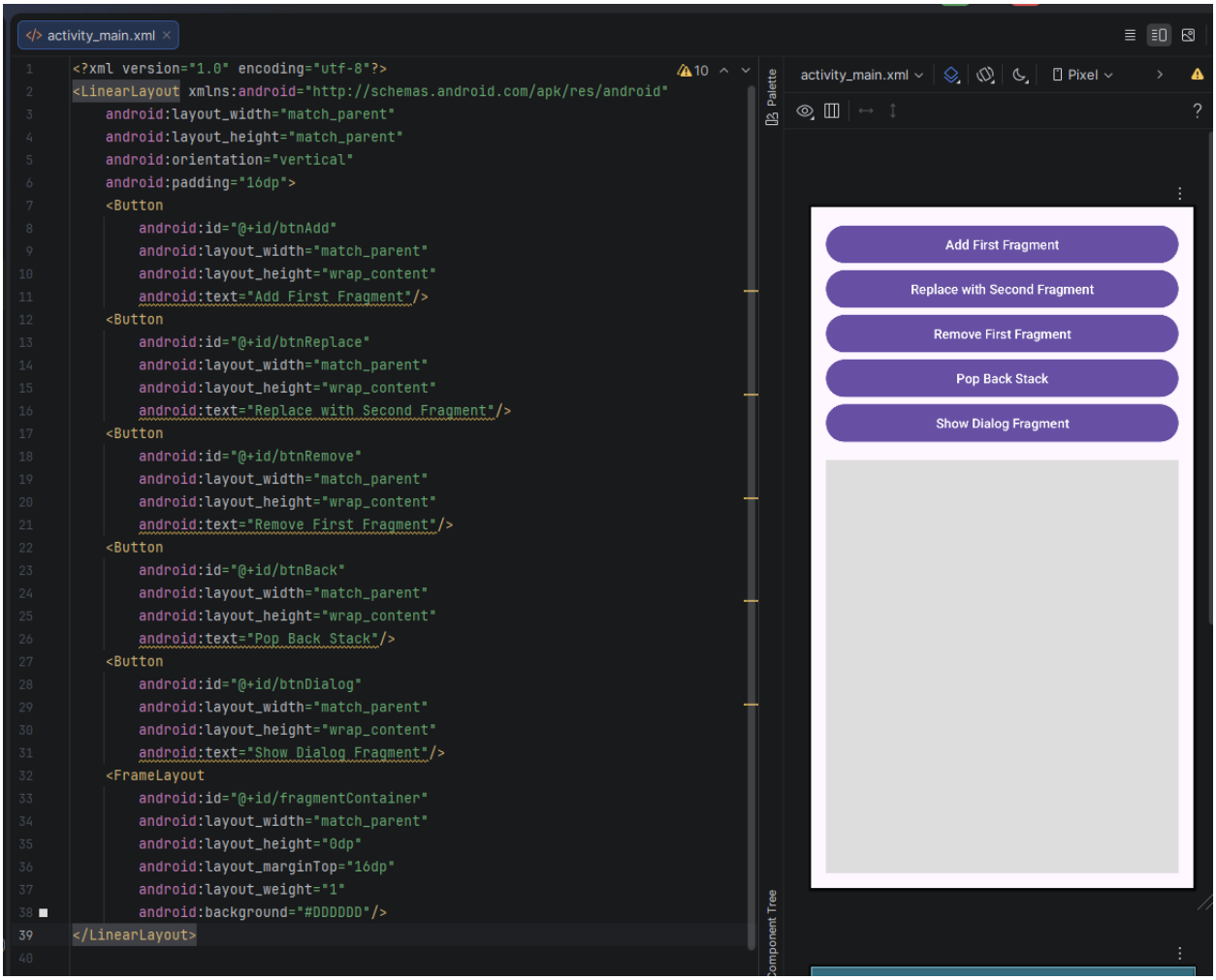
SecondFragment.java

```java
package com.example.lab_2;

import ...

public class SecondFragment extends Fragment {

    public SecondFragment() {
    }

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        return inflater.inflate(R.layout.fragment_second, container, attachToRoot: false);
    }
}
```

4.3 activity_main.xml

This layout contains:

- a Fragment container
- buttons to add, replace, remove, and pop back stack
- a button to open a dialog fragment



5. Add Fragment, Replace Fragment, Remove Fragment

In Android, Fragment operations are usually handled by:

- FragmentManager
- FragmentTransaction

Main methods:

- add() adds a Fragment
- replace() replaces the current Fragment with another
- remove() removes a Fragment
- addToBackStack() saves the transaction so the user can return

5.1 MainActivity.java

```java
MainActivity.java

public class MainActivity extends AppCompatActivity {

    2 usages
    Button btnAdd, btnReplace, btnRemove, btnBack, btnDialog;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnAdd = findViewById(R.id.btnAdd);
        btnReplace = findViewById(R.id.btnReplace);
        btnRemove = findViewById(R.id.btnRemove);
        btnBack = findViewById(R.id.btnBack);
        btnDialog = findViewById(R.id.btnDialog);

        btnAdd.setOnClickListener( View v -> {...});

        btnReplace.setOnClickListener( View v -> {...});

        btnRemove.setOnClickListener( View v -> {...});

        btnBack.setOnClickListener( View v -> {...});

        btnDialog.setOnClickListener( View v -> {...});
    }
}
```

```java
btnAdd.setOnClickListener( View v -> {
    FirstFragment firstFragment = FirstFragment.newInstance("Hello from Activity");

    getSupportFragmentManager() FragmentManager
            .beginTransaction() FragmentTransaction
            .add(R.id.fragmentContainer, firstFragment, tag: "FIRST_FRAGMENT")
            .addToBackStack( name: null)
            .commit();
});
```

```java
btnReplace.setOnClickListener( View v -> {
    SecondFragment secondFragment = new SecondFragment();

    getSupportFragmentManager() FragmentManager
            .beginTransaction() FragmentTransaction
            .replace(R.id.fragmentContainer, secondFragment)
            .addToBackStack( name: null)
            .commit();
});
```

```java
btnRemove.setOnClickListener( View v -> {
    Fragment fragment = getSupportFragmentManager().findFragmentByTag("FIRST_FRAGMENT");

    if (fragment != null) {
        getSupportFragmentManager() FragmentManager
                .beginTransaction() FragmentTransaction
                .remove(fragment)
                .commit();
    }
});
```

```java
btnBack.setOnClickListener( View v -> {
    getSupportFragmentManager().popBackStack();
});

btnDialog.setOnClickListener( View v -> {
    MyDialogFragment dialogFragment = new MyDialogFragment();
    dialogFragment.show(getSupportFragmentManager(), tag: "MY_DIALOG");
});
```

6. Fragment Pop Back Stack

The **Back Stack** stores Fragment transactions.

When you use:

.addToBackStack(null)

the transaction is saved. Then, when the user presses Back or when you call:

getSupportFragmentManager().popBackStack();

Android returns to the previous Fragment state.

Why Back Stack is important

Without back stack:

- replacing a Fragment permanently removes the previous one from the screen
- pressing Back may close the Activity instead of showing the previous Fragment

With back stack:

- users can move back through Fragment changes naturally

7. DialogFragment

A DialogFragment is a special type of Fragment used to display a dialog window.

It is better than using a normal dialog directly because:

- it is managed by FragmentManager
- it handles lifecycle better
- it survives configuration changes more safely

7.1 dialog_custom.xml

dialog_custom.xml

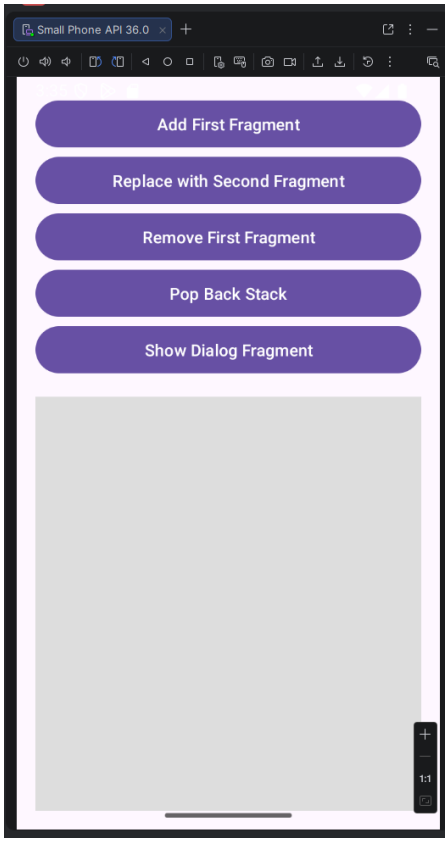
```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp">

    <TextView
        android:id="@+id/txtDialogMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is a DialogFragment"
        android:textSize="20sp"/>

</LinearLayout>
```

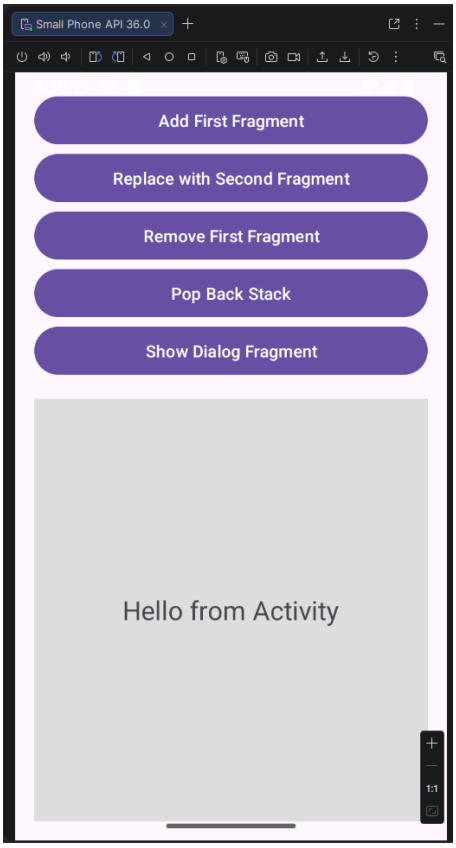
7.2 MyDialogFragment.java

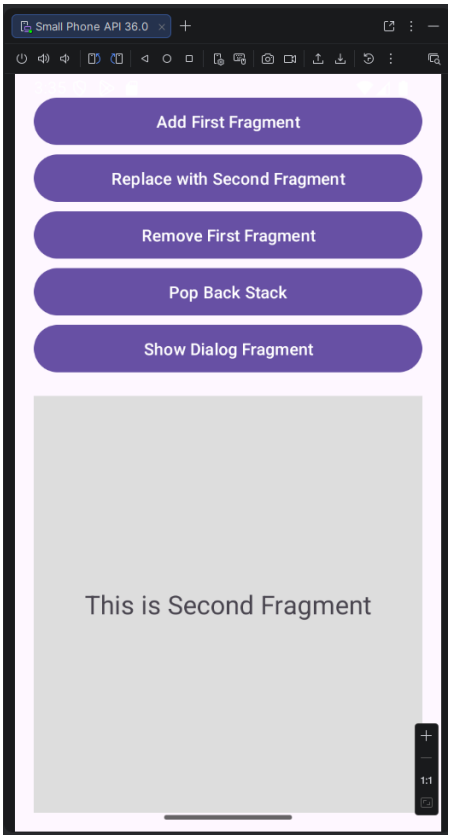
This will show a simple dialog when the button in MainActivity is pressed.

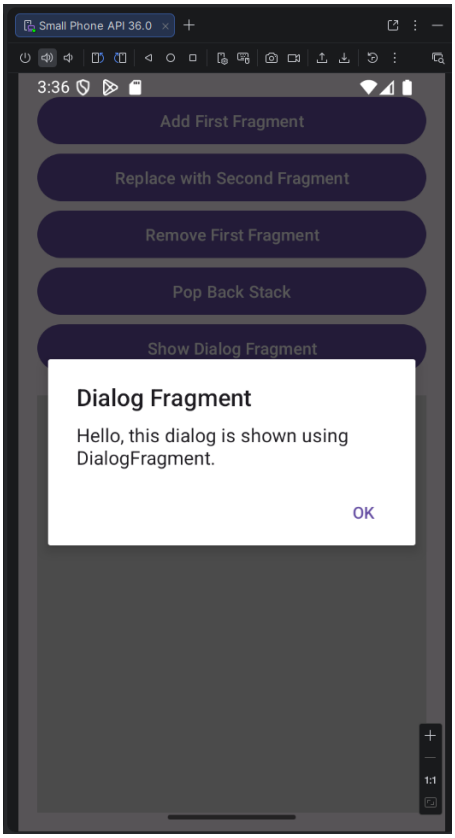
MyDialogFragment.java

```java
package com.example.lab_2;

import ...

public class MyDialogFragment extends DialogFragment {

    5 usages
    @NonNull
    @Override
    public Dialog onCreateDialog(Bundle savedInstanceState) {
        return new AlertDialog.Builder(requireActivity())
                .setTitle("Dialog Fragment")
                .setMessage("Hello, this dialog is shown using DialogFragment.")
                .setPositiveButton( text: "OK", ( DialogInterface dialog,  int which) -> dialog.dismiss())
                .create();
    }
}
```


Small Phone API 36.0
Add First Fragment
Replace with Second Fragment
Remove First Fragment
Pop Back Stack
Show Dialog Fragment


Small Phone API 36.0
Add First Fragment
Replace with Second Fragment
Remove First Fragment
Pop Back Stack
Show Dialog Fragment
Hello from Activity


Small Phone API 36.0
Add First Fragment
Replace with Second Fragment
Remove First Fragment
Pop Back Stack
Show Dialog Fragment
This is Second Fragment


Small Phone API 36.0
3:36
Add First Fragment
Replace with Second Fragment
Remove First Fragment
Pop Back Stack
Show Dialog Fragment
Dialog Fragment
Hello, this dialog is shown using DialogFragment.
OK

8. Pass Data from Activity to Fragment

A common way to pass data from Activity to Fragment is by using Bundle.

Example

In MainActivity:

FirstFragment firstFragment = FirstFragment.newInstance("Hello from Activity");

In FirstFragment:

```
public static FirstFragment newInstance(String message) {
    FirstFragment fragment = new FirstFragment();
    Bundle args = new Bundle();
    args.putString("message_key", message);
    fragment.setArguments(args);
    return fragment;
}
```

Then retrieve it in onCreateView():

```
if (getArguments() != null) {
    String message = getArguments().getString("message_key");
    txtMessage.setText(message);
}
```

This is the recommended pattern because:

- it keeps Fragment initialization clean
- Android can recreate the Fragment with its arguments later

9. Interaction Between Fragments

Fragments often need to interact with each other. Usually, they do not communicate directly. Instead, they communicate through the Activity.

For example:

- FirstFragment sends an event to MainActivity
- MainActivity updates SecondFragment

A common approach is to define an interface in the Fragment and let the Activity implement it.

Example: Fragment sends data to Activity

FirstFragment.java

```
import android.content.Context;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

public class FirstFragment extends Fragment {
    Button btnSend;
    public interface OnMessageSendListener {
        void onMessageSend(String message);
    }
    private OnMessageSendListener listener;
    public FirstFragment() {
    }


    @Override
    public void onAttach(@NonNull Context context) {
        super.onAttach(context);
        if (context instanceof OnMessageSendListener) {
            listener = (OnMessageSendListener) context;
        }
    }

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_first, container, false);
        btnSend = view.findViewById(R.id.btnSendData);
```

```
        btnSend.setOnClickListener(v -> {
            if (listener != null) {
                listener.onMessageSend("Message from FirstFragment");
            }
        });
        return view;
    }
}
```

Add button in fragment_first.xml

```
<Button
    android:id="@+id/btnSendData"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Send to Activity"/>
```

MainActivity.java

```
public class MainActivity extends AppCompatActivity implements
FirstFragment.OnMessageSendListener {

    @Override
    public void onMessageSend(String message) {
        SecondFragment secondFragment = new SecondFragment();

        Bundle bundle = new Bundle();
        bundle.putString("data", message);
        secondFragment.setArguments(bundle);

        getSupportFragmentManager()
                .beginTransaction()
                .replace(R.id.fragmentContainer, secondFragment)
                .addToBackStack(null)
                .commit();
    }
}
```

This shows how one Fragment can affect another through the Activity.